MyAnimLab

Rapport de test de charge et de test de sécurité

Date : 31 octobre 2026 • Branche : `ccr-e216aacd-7kbcw3`
Version testée : `d4e4911` • PHP 8.3 / MariaDB 10.11 / Apache 2.4 (mod_php, prefork)

1 Résumé

L'application a été déployée localement (Apache + mod_php 8.3, MariaDB 10.11, 4 vCPU, 16 Go) sur une base de démonstration volontairement gonflée à un volume réaliste d'animalerie, puis soumise à un test de charge et à un test de sécurité combinant analyse statique du code et exploitation dynamique.

La base de code est globalement saine : requêtes entièrement paramétrées (aucune injection SQL trouvée), jetons CSRF sur l'ensemble des traitements POST, session durcie, double authentification robuste, jetons de réinitialisation solides, téléversements confinés, aucune exécution de code dynamique. L'analyse n'a relevé *aucune* faille d'injection SQL, XSS stockée exploitable par un utilisateur non privilégié, ni exécution de commande.

Les points à corriger sont : (1) un **défaut de contrôle d'accès** sur la pharmacie permettant à un simple utilisateur de modifier le stock de médicaments, y compris stupéfiants *(vérifié dynamiquement)* ; (2) l'absence de contrainte d'unicité sur l'identifiant de connexion, ouvrant une **prise de contrôle de compte** ; (3) un **bogue fonctionnel** (erreur 500) qui casse l'ajout et la modification de médicaments ; (4) des pages qui **passent mal à l'échelle** avec le volume de données.

Sévérité	Critique	Élevée	Moyenne	Faible	Solide
Nombre	1	3	4	6	(voir §6)

2 Méthode et environnement

— **Déploiement** : Apache 2.4 avec `mod_php` 8.3 en MPM `prefork` (`MaxRequestWorkers` = 150), MariaDB 10.11 (`max_connections` = 151), sur la base de démonstration (`tools/demo_data.php`).
— **Volume de données** (injecté pour le test de charge) : 2 008 cages, 10 035 animaux, 162 008 contrôles de cage, 200 000 lignes de journal d'activité.
— **Outils** : `ab` (ApacheBench) et un générateur de charge multi-utilisateurs en Python (sessions authentifiées concurrentes) ; suite de tests du projet (`phpunit`, 257 tests : OK) ; `composer audit` (aucune vulnérabilité de dépendance) ; revue de code et exploitation manuelle.

3 Test de charge

3.1 Latence par page (volume réel, cache chaud, 1 utilisateur)

Page	Temps	Taille réponse
`export_data.php`	3 142 ms	2,4 Mo
`animal_register.php`	830 ms	**8,8 Mo**
`daily_round.php`	438 ms	**9,2 Mo**
`animal_search.php`	162 ms	0,6 Mo
`prescription_manage.php`	122 ms	1,1 Mo
`home.php`	116 ms	38 ko
(médiane des 98 pages : ≈9 ms)		

3.2 Débit et montée en charge (ApacheBench)

Page	Débit	p50	p99	Erreurs
`index.php` (anonyme)	1 294 req/s	–	183 ms	0
`get_notifications.php`	978 req/s	43 ms	194 ms	0
`cage_dash.php` (c=50)	79 req/s	625 ms	699 ms	0
`home.php` (c=100)	10 req/s	9 4 s	10 6 s	0
`daily_round.php` (c=50)	3 req/s	15 4 s	16 4 s	0

Charge mixte multi-utilisateurs (profil réaliste, 30 s par palier) : débit soutenu ≈75 req/s ; p95 passant de 559 ms (10 utilisateurs) à 9 8 s (200 utilisateurs), avec apparition d'erreurs 500 à 200 utilisateurs.

3.3 Constats

Élevée C1 — Épuisement des connexions MariaDB sous rafale. À 200 utilisateurs simultanés, `MaxRequestWorkers` (150) est atteint puis `max_connections` MariaDB (151) ; chaque requête PHP ouvre une connexion `mysqli` sans mutualisation. Le dépassement remonte en **erreur fatale non interceptée** (`Too many connections`, `dbcon.php:48`) servie en HTTP 500 au lieu d'une dégradation maîtrisée. *Recommandation* : aligner `MaxRequestWorkers` sur `max_connections` (le premier strictement inférieur au second), intercepter l'échec de connexion dans `dbcon.php` pour renvoyer une page 503 propre, et envisager un pool/connexions persistantes.

Élevée C2 — Pages volumineuses non paginées. `daily_round.php` et `animal_register.php` génèrent ≈9 Mo de HTML car elles rendent *tous* les animaux/cages sans pagination ; à 10 k animaux leur coût domine (3 à 15 s sous charge). *Recommandation* : pagination côté serveur et chargement à la demande.

Moyenne C3 — Sérialisation par verrou de session. Les requêtes d'une même session se sérialisent (verrou de fichier de session PHP). Comme chaque page interroge `get_notifications.php`, un onglet lent (p. ex. `daily_round`) bloque les requêtes concurrentes du même utilisateur. *Recommandation* : appeler `session_write_close()` tôt sur les pages en lecture seule (comme le fait déjà, correctement, le sondage de notifications).

4 Test de sécurité — vulnérabilités

Critique / Élevée S1 — Pharmacie accessible à tout utilisateur (vérifié)

`pharmacy_manage.php` ne vérifie que l'ouverture de session (`isset($_SESSION['username'])`), sans contrôle du rôle administrateur, alors que le guide réserve ces opérations aux administrateurs. **Exploitation vérifiée** : un compte de rôle `user` (`marc.dubois`, id 4) a ajusté le stock d'un lot (`adjustment -1.000`, `performed_by=4`, réponse 302). Un utilisateur peut aussi réceptionner des lots et, surtout, retirer le marquage « stupéfiant » d'un médicament, ce qui le fait **sortir du registre des stupéfiants** — impact réglementaire direct (BEA). *Correctif* : exiger le rôle `admin` pour `add_product`, `update_product`, `add_lot`, `start_use` et `adjust_stock`.

Élevée S2 — Prise de contrôle de compte par identifiant dupliqué (vérifié)

La colonne `users.username` n'a **aucune contrainte d'unicité** (vérifié : l'insertion d'un second compte avec `claire.martin@demo.fr` est acceptée). Le changement d'adresse dans le profil ne vérifie pas non plus l'unicité, et toutes les mises à jour sensibles (mot de passe, rôle, statut) sont indexées par `username` et non par `id`. **Scénario** : un utilisateur renomme son adresse en celle d'un administrateur, demande une réinitialisation, et le `UPDATE ... WHERE username=?` touche

les deux comptes. *Correctif* : `ALTER TABLE users ADD UNIQUE(username)` ; vérifier l'unicité et valider l'adresse (`FILTER_VALIDATE_EMAIL`) au changement de profil ; indexer toutes les écritures sur `id`.

Élevée S3 — Bogue cassant la pharmacie, erreur 500 (vérifié)

`pharmacy_manage.php` appelle `parse_number()`, définie dans `animal_lib.php`, **jamais inclus** par cette page. Tout envoi de formulaire `add_product` ou `update_product` provoque `Fatal error: Call to undefined function parse_number()` (HTTP 500, vérifié dans le journal Apache). L'ajout et la modification de médicaments sont donc **inopérants** (la démo ayant été peuplée par SQL direct, le défaut n'a pas été exercé). *Correctif* : `require_once 'animal_lib.php';` (ou `medication_lib.php`, qui en dépend) en tête de `pharmacy_manage.php`.

Moyenne S4 — Déverrouillage via « mot de passe oublié » et énumération (vérifié)

`forgot_password.php` remet `login_attempts=0` et `account_locked=NULL` à chaque demande : un attaquant non authentifié essaie 3 mots de passe, demande une réinitialisation (ce qui déverrouille), et recommence indéfiniment. Symétriquement, n'importe qui peut **bloquer** le compte d'autrui (déni de service, 3 échecs/15 min). De plus, les messages diffèrent selon l'existence du compte (*vérifié* : « Aucun utilisateur ne correspond » vs « Mot de passe incorrect »), permettant l'énumération. *Correctif* : ne pas toucher aux compteurs de verrouillage dans « mot de passe oublié » ; message générique unique ; limitation par IP.

Moyenne S5 — IDOR sur tâches et rappels

`manage_tasks.php` (édition/suppression) et `manage_reminder.php` agissent par `WHERE id=?` sans vérifier la propriété : un utilisateur peut modifier ou supprimer la tâche d'autrui, et le champ `assigned_by` provient du POST (usurpation d'expéditeur dans les notifications). *Correctif* : filtrer sur propriétaire/assigné/admin et toujours fixer `assigned_by` depuis la session.

Moyenne S6 — Changements de rôle sans invalidation de session

Le rôle est copié en session à la connexion ; une rétrogradation ou suppression (`manage_users.php`) ne tue pas les sessions actives : un administrateur déchu conserve ses droits jusqu'à expiration (30 min). *Correctif* : relire rôle/statut en base à chaque requête, ou invalider les sessions concernées.

Moyenne S7 — Injection de formule dans les exports CSV

`export_data.php`, `log_export.php` et les registres écrivent du texte utilisateur sans neutraliser les préfixes `= + - @` (exécution de formule à l'ouverture dans un tableur). *Correctif* : préfixer d'une apostrophe les cellules commençant par ces caractères.

Faible S8 — Divers (faible)

— En-têtes de sécurité (X-Frame-Options, CSP) absents des pages sans `header.php`, dont `index.php` (page de connexion).
— Changement d'adresse de profil sans re-saisie du mot de passe courant.
— Déconnexion en GET sans jeton ; formulaire de connexion sans jeton CSRF (atténué par `SameSite=Lax`).
— Compte administrateur par défaut amorcé (`schema.sql`) ; bannière d'avertissement seulement.
— `FILTER_SANITIZE_STRING` (déprécié en PHP 8.1) altère les apostrophes stockées.

— URL de souche (admin) non filtrée par schéma : `javascript:` possible (XSS stockée réservée à l'administrateur).
— `demo/*.php` directement accessibles (absents de la liste `.htaccess`) ; inoffensifs mais lèvent des erreurs.

5 Ce qui est solide

— **Injection SQL** : aucune. Toutes les requêtes à paramètres utilisateur sont préparées ; les identifiants dynamiques (ORDER BY, LIMIT, colonnes) proviennent de listes blanches ou d'entiers.
— **CSRF** : `csrf_valid()` en tête de ≈45 traitements POST ; actions destructrices en POST + jeton.
— **Contrôle d'accès** (hors S1/S5/S6) : pages d'administration et BEA correctement protégées ; points d'API (`get_task`, `get_notifications`) restreints à l'utilisateur ; scripts cron refusant l'appel non-CLI.
— **Téléversements** : liste blanche d'extensions + contrôle MIME `finfo`, noms aléatoires, dossiers interdits par `.htaccess`, téléchargement confiné par `realpath` (aucune traversée de chemin). Exposition des fichiers sensibles bloquée (*vérifié* : `.env`, `config.php`, `database/`, `vendor/` renvoient 403/404).
— **Authentification/session** : régénération de l'identifiant à la connexion, cookies HttpOnly/-SameSite/Secure, mode strict, expiration d'inactivité ; 2FA avec limite de tentatives, fenêtre de 5 min et anti-rejeu ; jetons de réinitialisation `random_bytes(32)`, 1 h, usage unique.
— **XSS** : échappement systématique (`htmlspecialchars`, `json_encode` avec drapeaux HEX).
— **Exécution de code** : aucun `exec`/`system`/`eval`/`unserialize`, aucun `include` à variable ; secrets non versionnés ; `composer audit` sans vulnérabilité.

6 Priorités de correction

1. **S3** (500 pharmacie) — correctif d'une ligne, remet la fonctionnalité en service.
2. **S1** (rôle pharmacie) et **S2** (unicité `username`) — impact réglementaire et prise de contrôle.
3. **C1** (épuisement connexions) et **S4** (déverrouillage) — robustesse et force brute.
4. **C2**, **S5**–**S7** — performance et cloisonnement.

Tests menés sur une base de démonstration jetable ; aucune donnée réelle n'a été touchée. Les mesures de charge reflètent un déploiement mono-serveur non optimisé et donnent des ordres de grandeur, non des garanties de capacité en production.